www.devmedia.com.br
[versão para impressão]
Link original: https://www.devmedia.com.br/sql-select-guia-para-iniciantes/29530

SQL Select: Guia para Iniciantes

Veremos nesse artigo um pequeno guia de consultas para o comando SELECT, um dos mais importantes da linguagem SQL.

Guia de consultas para o comando SELECT

A linguagem SQL foi criada com o objetivo de padronizar os comandos de manipulação de dados em SGBD's. Hoje em dia, apesar de a linguagem possuir uma quantidade considerável de extensões e implementações proprietárias, pode-se afirmar que a meta foi alcançada. Conhecendo bem a linguagem é possível acessar os recursos básicos de qualquer banco relacional, como Oracle, SQL Server ou MySQL, sem praticamente nenhuma mudança.

Saiba mais: Introdução prática ao comando SQL SELECT

Veremos nesse artigo um pequeno guia de consultas para o comando SELECT, um dos mais importantes da linguagem SQL.

SELECT simples

O **comando SELECT** permite recuperar os dados de um objeto do banco de dados, como uma tabela, view e, em alguns casos, uma stored procedure (alguns bancos de dados permitem a criação de procedimentos que retornam valor). A sintaxe mais básica do comando é:

```
SELECT <lista_de_campos>
FROM <nome_da_tabela></nome_da_tabela></lista_de_campos>
```

Exemplo:

```
SELECT CODIGO, NOME FROM CLIENTES
SELECT * FROM CLIENTES
```

O caractere * representa todos os campos. Apesar de prático, este caractere não é muito utilizado, pois, para o SGBD é mais rápido receber o comando com todos os campos explicitados. O uso do * obriga o servidor a consultar quais são os campos antes de efetuar a busca dos dados, criando mais um passo no processo.

COMANDO WHERE

A cláusula Where permite ao comando SQL passar condições de filtragem. Veja o exemplo da **Listagem 1**.

```
SELECT CODIGO, NOME FROM CLIENTES
WHERE CODIGO = 10
SELECT CODIGO, NOME FROM CLIENTES
WHERE UF = 'RJ'
SELECT CODIGO, NOME FROM CLIENTES
WHERE CODIGO >= 100 AND CODIGO <= 500
SELECT CODIGO, NOME FROM CLIENTES
WHERE UF = 'MG' OR UF = 'SP'
```

Listagem 1. Exemplo where

Os parênteses corretamente utilizados dão mais poder as consultas, conforme exemplo abaixo:

```
SELECT CODIGO, NOME FROM CLIENTES
WHERE UF = 'RJ' OR (UF = 'SP' AND ATIVO = 'N')
```

Neste comando todos os clientes do Rio de Janeiro e apenas os clientes inativos de São Paulo seriam capturados.

Agora veja o exemplo a seguir:

```
SELECT CODIGO, NOME FROM CLIENTES
WHERE (ENDERECO IS NULL) OR (CIDADE IS NULL)
```

Aqui, todos os clientes que não possuem endereço ou cidade cadastrada serão selecionados.

FILTRO DE TEXTO

Para busca parcial de string, o SELECT fornece o operador LIKE. Veja o exemplo abaixo:

```
SELECT CODIGO, NOME FROM CLIENTES
 WHERE NOME LIKE ‘MARIA%’
```

Neste comando, todos os clientes cujos nomes iniciam com Maria serão retornados. Se quisermos retornar os nomes que contenham ‘MARIA’ também no meio, podemos alterar para o exemplo a seguir:

```
SELECT CODIGO, NOME FROM CLIENTES
WHERE NOME LIKE ‘%MARIA%’
```

O uso de máscara no início e no fim da string fornece maior poder de busca, mas causa considerável perda de performance. Este recurso deve ser utilizado com critério.

Uma observação*: em alguns bancos de dados, a máscara de filtro não é representada por %. Consulte a referência do banco para verificar o caractere correto.*

Por padrão, a SQL diferencia caixa baixa de caixa alta. Para eliminar essa diferença, utiliza a função UPPER. Veja abaixo:

```
SELECT CODIGO, NOME FROM CLIENTES
WHERE UPPER(NOME) LIKE ‘MARIA %SILVA%’
```

ORDENAÇÃO

A ordenação pode ser definida com o comando ORDER BY. Assim como no comando WHERE, o campo de ordenação não precisa estar listado como campo de visualização. Veja o exemplo da **Listagem 2.**

```
SELECT CODIGO, NOME FROM CLIENTES
ORDER BY NOME
SELECT CODIGO, NOME FROM CLIENTES
ORDER BY UF, NOME
```

Listagem 2. Exemplo order by

A utilização da palavra DESC garante a ordenação invertida:

```
SELECT CODIGO, NOME FROM CLIENTES
ORDER BY NOME DESC
SELECT CODIGO, NOME FROM CLIENTES
ORDER BY UF DESC
```

JUNÇÃO DE TABELAS

O SELECT permite juntar duas ou mais tabelas no mesmo resultado. Isso pode ser feito de várias formas. Uma delas segue abaixo:

```
SELECT CLIENTES.CODIGO, CLIENTES.NOME, PEDIDOS.DATA
FROM CLIENTES, PEDIDOS
WHERE CLIENTES.CODIGO = PEDIDOS.CODCLIENTE
```

Nesta linha as tabelas relacionadas CLIENTES e PEDIDOS são unificadas através do campo chave, em uma operação de igualdade. Repare que os nomes dos campos passam a ser prefixados pelo nome das tabelas, resolvendo duplicidades. Uma versão resumida desse comando pode ser como abaixo:

```
SELECT A.CODIGO, A.NOME, B.DATA, B.VALOR
FROM CLIENTES A, PEDIDOS B
WHERE A.CODIGO = B.CODCLIENTE
```

O uso de aliases no código SQL torna a manutenção mais simples.

No comando abaixo temos várias tabelas unificadas em uma mesma cláusula. Veja a **Listagem 3.**

```
SELECT A.CODIGO, A.NOME, B.DATA, B.VALOR, C.QTD, D.DESCRIC
FROM CLIENTES A, PEDIDOS B, ITENS C, PRODUTOS D
 WHERE A.CODIGO = B.CODCLIENTE
 AND B.CODIGO = C.CODPEDIDO
 AND C.CODPRODUTO = D.CODIGO
```

Listagem 3. Exemplo de tabelas unificadas

Neste comando unificamos as tabelas relacionadas CLIENTES, PEDIDOS, ITENS e PRODUTOS. Veja mais este exemplo:

```
SELECT A.CODIGO, A.NOME, B.DATA, B.VALOR
FROM CLIENTES A, PEDIDOS B
WHERE A.CODIGO = B.CODCLIENTE
AND A.UF = ‘RJ’
```

Observe que a junção através da igualdade de campos traz como resultado somente os registros que possuem referências nas duas tabelas. Observe o comando abaixo:

```
SELECT A.CODIGO, A.DESCRICAO, B.DESCRICAO
FROM PROUTOS A, COMPONENTES B
WHERE A.CODIGO = B.CODPRODUTO
```

Os produtos que não possuem componentes não são selecionados, caso seja necessário criar uma listagem incluindo também os registros que não possuem correspondência, deve-se utilizar o comando JOIN.

COMANDO JOIN

A junção de tabelas no comando SELECT também pode ser feita com o comando JOIN. Este comando deve ser utilizado com a palavra reservada INNER ou com a palavra OUTER:

- INNER: Semelhante ao uso do operador “=” na junção de tabelas. Aqui os registros sem correspondências não são incluídos. Esta cláusula é opcional e pode ser omitida no comando JOIN.

- OUTER: Os registros que não se relacionam também são exibidos. Neste caso, é possível definir qual tabela será incluída na seleção, mesmo não tendo correspondência.

Para exemplificar, temos as tabelas abaixo:

Observe os exemplos e o resultado produzido:

```
SELECT A.CODIGO, A.DESCRICAO, B.DESCRICAO, B.QTD
FROM PRODUTOS A
INNER JOIN COMPONENTES B
ON (A.CODIGO = B.CODPRODUTO)
```

Este comando pode ser escrito na versão resumida abaixo:

```
SELECT A.CODIGO, A.DESCRICAO, B.DESCRICAO
FROM PRODUTOS A
JOIN COMPONENTES B
ON (A.CODIGO = B.CODPRODUTO)
```

Como mostrado no resultado 1, os produtos que não possuem componentes não são incluídos na seleção.

```
SELECT A.CODIGO, A.DESCRICAO, B.DESCRICAO, B.QTDE
FROM PRODUTOS A
LEFT OUTER JOIN COMPONENTES B
ON (A.CODIGO = B.CODPRODUTO)
```

Neste comando todos os produtos serão incluídos na seleção, independente de possuírem um componente. Observe que a palavra LEFT se refere à primeira tabela do relacionamento. O mesmo comando poderia ser descrito como na **Listagem 4.**

```
SELECT A.CODIGO, A.DESCRICAO, B.DESCRICAO
FROM COMPONENTES A
RIGHT OUTER JOIN PRODUTOS B
ON (A.CODIGO = B.CODPRODUTO)
```

Listagem 4. Exemplo da palavra left

A ordem das tabelas foi invertida mas o resultado é o mesmo. Observe mais alguns exemplos:

```
SELECT A.CODIGO, A.DESCRICAO, B.DESCRICAO, B.QTDE
FROM PRODUTOS A
JOIN COMPONENTES B
ON (A.CODIGO  = B.CODPRODUTO)
WHERE A.CATEGORIA = 1
```

Agora veja o código q reproduz o resultado 4:

```
SELECT A.CODIGO, A.DESCRICAO, B.DESCRICAO
FROM PRODUTOS A JOIN COMPONENTES B
ON (A.CODIGO = B.CODPRODUTO)
WHERE A.CATEGORIA = 1 OR A.CATEGORIA = 2
ORDER BY A.CATEGORIA, A.DESCRICAO
```

FULL OUTER JOIN

Podemos ainda combinar o uso de INNER e OUTER através do comando FULL OUTER JOIN. Neste caso, todos os registros das duas tabelas envolvidas serão exibidos, tendo ou não relacionamento. Observe:

UNION

Existe ainda uma segunda forma de juntar **tabelas com o comando SELECT**. Através do parâmetro UNION, é possível colar o conteúdo de duas tabelas. Veja o exemplo:

```
SELECT CODIGO, NOME FROM CLIENTES
UNION
SELECT CODIGO. NOME FROM FUNCIONARIOS
```

O resultado deste comando é a listagem de todos os clientes e a listagem de todos os funcionários, dentro do mesmo result set. Repare que no comando JOIN á união é horizontal e no UNION a união é vertical.

Por default, os registros duplicados são eliminados na cláusula UNION. No exemplo anterior, se tivéssemos um cliente com o mesmo nome e código de um funcionário, apenas o registro da primeira tabela seria exibido. Para incluir todos os registros, independente de duplicidade, utilize a palavra ALL:

```
SELECT CODIGO, NOME FROM CLIENTES
UNION ALL
SELECT CODIGO, NOME FROM FUNCIONARIOS
```

FUNÇÕES DE AGRUPAMENTO

São cinco as funções básicas de agrupamento:

- AVG: Retorna a média do campo especificado

- SELECT AVG(VALOR) FROM PEDIDOS
- MIN/MAX/SUM: Respectivamente retorna o menor valor, o maior e o somatório de um grupo de registros:
- SELECT MIN(VALOR) FROM PEDIDOS
- SELECT MAX(VALOR) FROM PEDIDOS
- SELECT AVG(VALOR) FROM PEDIDOS
- COUNT: Retorna a quantidade de itens da seleção

Um exemplo de uso é o que pode ser visto abaixo:

```
SELECT COUNT(CODIGO) FROM CLIENTES
```

AGRUPAMENTO

Um poderoso **recurso do comando SELECT** é o parâmetro GROUPY BY. Através dele podemos retornar informações agrupadas de um conjunto de registros, estabelecendo uma condição de agrupamento. É um recurso muito utilizado na criação de relatórios. Para exemplificar, temos as tabelas CLIENTES E PEDIDOS a seguir.

```
SELECT CODCLIENTE, MAX(VALOR)
FROM PEDIDOS
GROUP BY CODCLIENTE
```

O comando acima retorna o maior valor de pedido de cada cliente. Observe o resultado:

```
SELECT CODCLIENTE, COUNT(*)
FROM PEDIDOS
GROUPY BY CODCLIENTE
```

Abaixo vemos quantos pedidos foram feitos por cada cliente.

HAVING

Através do comando HAVING podemos filtrar a cláusula GROUP BY. Observe o comando abaixo:

```
SELECT CODCLIENTE, COUNT(*)
FROM PEDIDOS
GROUPY BY CODCLIENTE
HAVING COUNT(*) >= 2
```

Somente os clientes com 2 ou mais pedidos serão selecionados. Repare que o HAVING é utilizado, geralmente com alguma função de agrupamento. Para filtros normais, pode-se utilizar o comando WHERE. Observe o exemplo abaixo:

```
SELECT CODCLIENTE, COUNT(*)
FROM PEDIDOS
WHERE DATA > ‘06/10/2002’
GROUPY BY CODCLIENTE
HAVING COUNT(*) >= 2
```

Repare que o cliente número 3 apresentou apenas dois pedidos, visto que o primeiro não possui data maior que 06/10.

Com esse artigo voltado ao público mais iniciante em SQL conseguimos solucionar muitas dúvidas sobre como efetuar consultas **usando o comando Select** e você aprende a adicionar todas as Cidades e Estados do Brasil em um banco de dados usando SQL.

Relacionados a Programação

- JavaScript Replace: substituindo valores em uma string:

  Conheça o método replace() e aprenda a substituir uma sequência de caracteres por outra. Como substituir valores em uma string com JavaScript Replace.
- PostgreSQL: deixando seu banco de dados seguro e rápido:

  Este artigo traz uma abordagem prática, contextualizando problemas de performance e segurança, abordando soluções cotidianas que visam resolver os principais problemas durante a configuração do banco de dados.
- Levantamento de requisitos: Planejando uma aplicação de lembretes:

  Neste curso você verá como funciona uma reunião para levantamento de requisitos de um novo software. Planejaremos aqui uma aplicação web de lembretes, em que o usuário pode se cadastrar, logar e gerenciar seus lembretes.

Saiba mais sobre SQL ;)

- Curso de SQL:

  A linguagem SQL é amplamente utilizada em diversos tipos de aplicações que utilizem bancos de dados relacionais.
- Guias Banco de Dados:

  Aqui você encontra o Guia de estudo ideal para aprimorar seus conhecimentos nos principais Banco de Dados do mercado. Escolha o seu e bons estudos!
- Banco de Dados para Programadores:

  Neste guia você encontrará os principais conteúdos que você precisa estudar, como desenvolvedor, para trabalhar com bancos de dados.